

1. Global vs. Local Insight

Feature	Global Importance (BotAttack)	Global Importance (Normal)	Global Importance (PortScan)	Interpretation
Port	Highest	Highest	Lowest	Across the three classes Port has the strongest mean absolute SHAP effect. The random-forest learns complicated interactions on the port column, likely because certain ports (e.g., 443, 80, 22) are more common for benign traffic while others (e.g., 4444, 6667, 31337) are associated with scans or attacks.

Feature	Global Importance (BotAttack)	Global Importance (Normal)	Global Importance (PortScan)	Interpretation
PayloadSize	Medium	Medium	Highest	All classes are sensitive to the size of the payload – large values usually indicate normal HTTP/HTTPS requests, while very small or very large values may be a sign of probing. The decision trees only use these categorical variables rarely, suggesting that the port and payload are the main discriminators.
Protocol / Request_Type	Very low	Very low	Very low	

	Global Importance (BotAttack)	Global Importance (Normal)	Global Importance (PortScan)	Interpretation
Status	Low	Low	Low	Success vs. failure of a request plays a secondary role; it can help confirm whether a cookie-request is actually a handshake or a failed scan.
User_Agent	Low	Very low	Very low	The user-agent contributes little on average, implying that the agent string is not a strong signal for attack versus normal.

Local Observations

Taking the six local SHAP samples:

- **BotAttack** predictions (instances 6094, 8158, 8058, 7222) are dominated by *Port* and *Payload_Size* contributions. The sign patterns are consistent: a *large* payload and a *normal* port generate negative values for BotAttack,

whereas a *small* payload on a *high port* (e.g., 443) pushes the value toward normal.

- **PortScan** predictions almost never involve *User_Agent* or *Status*. The Port just slightly nudges the probability, but payload size is the primary differentiator (negative contribution when payload falls in the -0.84 to -0.03 range).
- **Normal** predictions are reflected by a positive *Port* contribution (often 0.05–0.2) and a positive *Payload_Size* contribution, with very small adjustments from *Request_Type* and *User_Agent*.

These patterns line up with the global ranking—*Port* is always top, followed by *Payload_Size*, while the remaining features provide fine-tuning.

2. Feature Influence on Attack Detection

- **Primary Driver:** *Port*. The random forest relies heavily on port number to separate attack from normal; typical ports used by scans (e.g., 21, 22, 23) are far more likely to be flagged.
- **Secondary Driver:** *Payload_Size*. Small or extremely large payloads trigger the model to lean towards attack or scan states.
- **Tertiary Drivers:** *Request_Type*, *Protocol*, *Status*, *User_Agent* contribute only marginally, primarily refining the decision boundary after the first two features have been considered.

The feature importance hierarchy can be verbally expressed as:

Port > Payload_Size > Request_Type Protocol Status User_Agent.

3. Dataset Patterns & Correlations

1. Port-Payload Correlation

- Normal traffic typically falls on well-known ports (80, 443, 53) and has a medium payload (0 ± 0.5 after scaling).
- Attacks/Scans either target low ports (21–23 for FTP/Shell) or high ports (4444, 6667), with an accompanying low or very large payload.

2. Request_Type Distribution

- The majority of normal instances are HTTP/HTTPS (2–3). Scans use a mix of DNS (0) and SSH/Telnet (5–6).

- However, the global importance for this feature is low, suggesting that the model does not rely on request type alone.

3. Status (Success/Failure) Tends to Shift

- Failure responses (0) correlate slightly more with scans, but the effect size is minor; many normal scans still succeed.

4. User_Agent

- Some malicious scanners use agent strings like Nikto (1), Kali (3) or nmap (4), but their contribution is negligible, implying that the port/payload pattern is stronger.

Overall, the data displays a clear **port-based attack signature**: low or high port numbers combined with atypical payload sizes.

4. Analysis of Prediction Samples

The provided predictions show a perfect match between true and predicted labels—only the BotAttack (index 0) and PortScan (index 2) instances are correctly labeled. No misclassifications are detected in this trimmed sample.

Potential Discrepancies in Full Test Set

- **False Negatives**: If the random forest mislabels a scan on a common port (e.g., port 80) with a normal payload, the model might predict Normal. This would happen if a scanning tool masquerades as a normal web request.
- **False Positives**: Rarely, a legitimate FTP login on port 21 with a large payload might be flagged as BotAttack if the model over-reliance on port triggers an attack decision.

These edge cases underscore a **trade-off** between sensitivity and specificity: raising port-thresholds improves recall of attacks but may degrade precision.

5. Impact of Categorical Encodings

- The encoding scheme (0-13 for ports, 0-6 for request types, etc.) forces the random forest to learn linear boundaries between encoded values, but trees can still separate categories regardless of numeric distance.
- Mis-encoding may cause the model to treat adjacent ports sharply different (port 5 vs 6) even though traffic patterns are similar.
- Categorical features with many levels (e.g., Port has 14 levels) can lead to many “splits” in trees; in the feature importance values every split counts

as a contribution, which explains why *User_Agent* appears low—few trees use that split.

A more informative encoding could be **label-frequency weighting** or **target encoding** for ports, preserving the notion that truly similar ports (80, 443) share attributes.

6. Cybersecurity Insights

1. Port-Based Attack Signature

- High-confidence attack alerts can be triggered by detecting unusual activity on high-risk ports (21, 22, 23, 4444, 6667, 31337).
- Combining this with an abnormal payload size sharpens the alarm.

2. Low-Quality User-Agents

- Although the model gives them little weight, many scans use generic or malicious agents. An external rule-engine could flag such agents as low-confidence alerts.

3. Failure Status as an Indicator

- While weak, a cluster of consecutive failed requests on a single port might suggest a port-scan, a signal that could be flagged by a threshold on status.

4. Dynamic Policy Adjustment

- The pipeline already uses SMOTE for balancing. Regular re-training with new samples of emerging bot signatures will sustain accuracy.

These findings are actionable in an IDS or SIEM: add **port-payload** correlators and subtle **status-based** heuristics to improve detection rates.

7. Recommendations for Improvement

Area	Suggested Action
Feature Engineering	• Add a time-of-day or temporal feature (e.g., hourly frequency). • Engineer a composite feature: Port_Risk_Score based on known malicious port lists.
Encoding	• Apply target encoding on Port and Request_Type to capture class-specific frequency patterns.

Area	Suggested Action
Model Enhancements	• Use deeper trees or a higher number of trees to enable finer decision boundaries for rare port attacks. • Incorporate gradient-boosted trees (keeping model type limitation in mind).
Threshold Calibration	• Post-process SHAP scores to calibrate class thresholds for a desired precision–recall trade-off.
Explainability Integration	• Build an automated rule-extractor from SHAP + LIME patterns to produce human readable policy suggestions.
Dataset Expansion	• Gather more samples of obscure ports (e.g., 3306, 5432) and emerging bot signatures to avoid concept drift.

By following these steps, the RandomForest model will maintain its state-of-the-art accuracy while increasing interpretability and adaptability to new bot/scan techniques.